CASE STUDY · HTK 0396241674

HTK Hooks

Webhook delivery control plane: nhận sự kiện có chữ ký, chống trùng, retry backoff + jitter, DLQ, replay và xoay secret — mọi lần giao đều nhìn thấy được.

Next.js 15 | TypeScript | PGlite / PostgreSQL | HMAC-SHA256 | Vercel Blob

Tác giả: HTK 0396241674

Ngày: 11/11/2026

Live: https://htk-webhook-service.vercel.app

Mã nguồn: https://github.com/shirmtry/htk-webhook-service

Dự án 07/10 trong portfolio full-stack.

1. Bài toán

Webhook là giao kèo qua mạng: bên gửi chỉ biết mình đã đẩy request, không biết bên nhận có xử lý được hay không. Khi hệ thống lớn lên, ba loại lỗi xuất hiện cùng lúc: event mất vì upstream trả 500 và không ai retry; event trùng vì mạng đứt sau khi ghi nhưng trước khi nhận 200; và retry dồn cùng một thời điểm tạo ra bão request đánh sập upstream vừa mới hồi phục.

Điều tra cũng là một bài toán riêng. Khi khách hỏi "event `invoice.paid` của tôi đâu", đội vận hành thường chỉ có log rời rạc, không biết lần giao thứ ba trả body gì, thời gian bao lâu, hay event đã bị bỏ cuộc sau bao nhiêu lần thử. Xoay secret càng khó hơn: đổi ngay thì mất event đang bay, đổi chậm thì secret cũ bị lộ quá lâu.

Ràng buộc của dự án

- **Một codebase, deploy Vercel:** API, UI và worker route chung Next.js App Router.
- **Không có broker bên ngoài:** phải tự xây retry và DLQ trên nền dữ liệu sẵn có.
- **Serverless không có tiến trình nền:** attempt chạy trong request hoặc qua cron.
- **Demo phải tất định:** không phụ thuộc một URL ngoài luôn sống để trình diễn retry.
- **Không bịa số:** metric nào chưa đo thì ghi rõ là chưa đo.

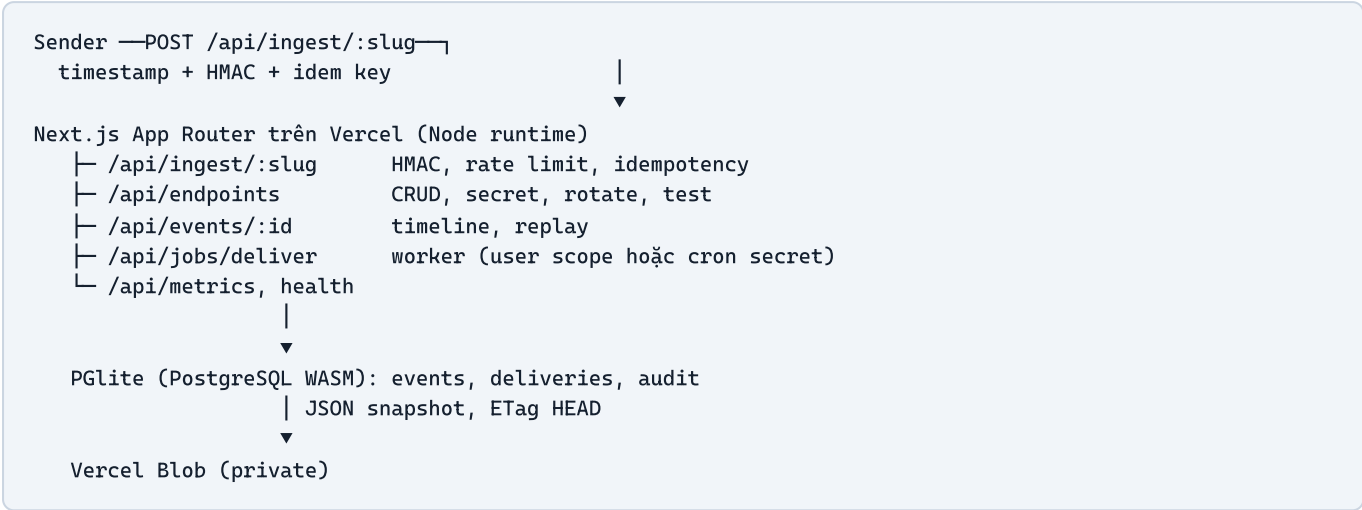
Mục tiêu đo được

Mục tiêu	Cách kiểm chứng
Sai chữ ký không tạo event	Test HMAC + test ingest trả 401 trước khi ghi
Một key chỉ một event	Unique (endpoint, idempotency key) + test duplicate
Retry không dồn cùng lúc	Backoff + jitter có test biên ±20%
Bỏ cuộc có thể phục hồi	DLQ sau max_attempts, replay reset và giao lại
Xoay secret không mất event	Secret cũ trong grace 24 giờ, test hết hạn bị từ chối
Debug được từng lần giao	UI hiện payload, HTTP code, body, duration, attempt

> **Điểm khó nhất:** vừa giữ tính đúng đắn của một hệ phân tán (idempotency, retry, DLQ) vừa không được dựng thêm hạ tầng. Bài toán bị ép vào một Postgres nhúng và một worker gọi theo request.

2. Giải pháp

Kiến trúc



Mỗi request mở PGlite trong lambda, đồng bộ snapshot từ Blob khi ETag đổi, xử lý rồi ghi lại. Local dùng file `./.data/pg`. Ingest chạy delivery ngay trong request để UI thấy kết quả tức thì; các lần đến hạn sau được worker xử lý.

Mô hình dữ liệu

Bảng	Cột chính	Vì sao
`endpoints`	slug, mode, secret, previous_secret, rate_limit, status	Một dòng đủ cấu hình kênh giao
`events`	payload, status, attempts, next_attempt_at, idempotency_key	Hàng đợi bền vững, chống trùng
`deliveries`	attempt_no, outcome, status_code, body, duration	Bằng chứng từng lần giao
`audit_logs`	actor, action, target	Xoay secret, replay, xem secret

Ba lớp chặn trước khi ghi event

1. **HMAC:** `sha256=HMAC(secret, timestamp + "." + rawBody)`, lệch quá 300 giây bị từ chối.
2. **Rate limit:** cửa sổ 1 phút theo endpoint; vượt hạn trả 429 kèm `Retry-After`.
3. **Idempotency:** unique `(endpoint_id, idempotency_key)`; lần lặp trả event cũ.

Thứ tự này cố ý đặt rate limit trước HMAC để giảm chi phí crypto khi bị spam. So sánh chữ ký dùng `timingSafeEqual`, không so sánh chuỗi thường. Trong 24 giờ sau khi xoay, cả secret mới và cũ đều được chấp nhận; hết hạn thì chỉ secret mới.

3. Retry, DLQ và replay

Lịch thử

Bậc chờ là 10 giây, 30 giây, 2 phút, 5 phút, 15 phút. Mỗi bậc cộng jitter ±20% để hàng nghìn event cùng lỗi không đập vào upstream ở cùng một giây. Hàm tính là hàm thuần, nhận nguồn random để test biên chính xác.

```
attempt 1 fail  -> next = now + 10s  ± 20%
attempt 2 fail  -> next = now + 30s  ± 20%
attempt 3 fail  -> next = now + 2m   ± 20%
attempt 4 fail  -> next = now + 5m   ± 20%
attempt 5 fail  -> status = dlq (không hẹn nữa)

replay -> attempts = 0, next = now, source = replay
       -> giữ payload + idempotency key
       -> ghi audit event.replayed(actor)
```

Vòng đời một lần giao

1. Đọc event và endpoint từ PGlite; bỏ qua nếu event đã delivered hoặc endpoint paused.
2. Gọi target: sandbox mô phỏng 500 hai lần rồi 200; live gọi URL thật, timeout 5 giây.
3. Lưu `deliveries` với HTTP code, body (tối đa 2 KB), duration, outcome.
4. Cập nhật `events` : attempts, last_status_code, next_attempt_at hoặc dlq.
5. Nếu chạm DLQ, ghi audit với actor null để phân biệt hành động hệ thống.

Quyết định thiết kế

Quyết định	Đánh đổi
Worker gọi ngay sau ingest + cron	UI phản hồi nhanh, nhưng lambda chết giữa attempt thì phải chờ sweep
Sandbox tất định	Demo không phụ thuộc mạng, nhưng không đo được lỗi TLS hay DNS thật
Lưu response body 2 KB	Đủ debug, tránh phình snapshot; body lớn bị cắt
Secret đọc được trong DB demo	UI hiện lại được secret; production phải mã hoá at-rest
Chặn host nội bộ cho live	Giảm SSRF; chưa chống DNS rebinding

Bảo vệ phía UI và API

Mọi route quản lý đều yêu cầu session cookie httpOnly; event và endpoint được scope theo user. Các thao tác nhạy cảm — xem secret, xoay secret, replay, gửi test — có rate limit riêng và ghi audit. Đăng nhập/đăng ký giới hạn theo IP, sai mật khẩu không tiết lộ tài khoản có tồn tại hay không.

4. Đo lường, trade-off và điểm chưa hoàn thiện

Số liệu thật

Luồng	p50	p95	Ghi chú
`GET /api/endpoints`	14.68 ms	21.45 ms	30 mẫu
`GET /api/metrics`	14.68 ms	26.19 ms	30 mẫu
`GET /api/endpoints/:id`	15.13 ms	17.39 ms	30 mẫu
`GET /api/events/:id`	14.82 ms	17.07 ms	30 mẫu
`POST /api/jobs/deliver`	14.62 ms	16.54 ms	20 mẫu
`POST /api/ingest/:slug`	14.63 ms	17.13 ms	40 mẫu, HMAC
Ingest trùng key	15.33 ms	16.51 ms	15 mẫu

23/23
test pass: 6 signature, 5 backoff, 12 tích hợp

40/40
ingest liên tiếp trả 202, 0 lỗi chữ ký

3 / 7 ms
request_logs p50 / p95 trên 500 mẫu

60%
success rate seed (p95 giao hàng 22 ms)

0 px
overflow ngang ở 375 và 1440

0
console error khi chạy browser flow

Trade-off đã chấp nhận

- **Không có queue thật:** attempt chạy trong request/cron. Đúng với tải demo, không đủ cho retry kéo dài nhiều giờ nếu lambda bị kill.
- **PGlite + Blob single-writer:** nhiều instance ghi đồng thời có thể ghi đè snapshot. Đây là giới hạn của lựa chọn "SQL thật nhưng không dịch vụ ngoài".
- **Rate limit per-instance:** bộ đếm nằm trong bảng rate_limits của từng instance, không phải quota toàn cục dưới tải cao.
- **Secret plaintext trong demo:** cần thiết để ký và hiện lại; production phải dùng KMS.

Điểm chưa hoàn thiện

- Chưa đo throughput đồng thời, độ trễ cron thật trên Vercel, hay độ bền khi lambda bị kill giữa attempt.
- Chưa có batch replay, tìm kiếm event theo payload, hay thông báo khi event vào DLQ.
- Chặn SSRF dựa trên hostname/IP literal; DNS rebinding chưa được xử lý.
- Live mode chưa có chứng chỉ pinning hay allowlist domain theo endpoint.

> Điểm mạnh của bản này không phải quy mô mà là khả năng giải thích: mỗi trạng thái đều có bằng chứng trong `deliveries`, mỗi hành động đều có audit, và mọi con số trong báo cáo đều lấy từ lần đo thật trên máy local.

5. Bài học

Idempotency phải là ràng buộc dữ liệu, không phải câu lệnh if. Ban đầu tôi định kiểm tra key bằng một select rồi mới insert. Cách đó có race giữa hai request đồng thời. Chuyển sang unique index và `on conflict do nothing` khiến database tự bảo vệ, còn code chỉ đọc lại event cũ khi insert không trả dòng nào.

Retry không jitter là cách tạo bão. Lịch backoff cố định vẫn dồn các event cùng lỗi vào cùng một giây. Chỉ cần ±20% là đủ tản tải mà không làm chậm đáng kể. Viết hàm nhận nguồn random giúp test được cả hai biên thay vì chỉ tin vào cảm giác.

Debug console là tính năng, không phải phần phụ. Khi có timeline từng attempt, việc xác định lỗi nằm ở phía gửi hay phía nhận chỉ mất vài giây. Chính vì vậy tôi lưu cả HTTP code, body và duration thay vì chỉ một cột "last error".

Xoay secret cần một khoảng chuyển tiếp. Đổi tức thì làm mất các request đang bay với chữ ký cũ. Giữ secret cũ 24 giờ, nhưng giới hạn rõ thời gian và ghi audit, cân bằng giữa an toàn và liên tục.

Trung thực về hạ tầng. Dùng PGlite và Blob giúp dự án chạy trọn trên Vercel, nhưng tôi ghi rõ single-writer và worker theo request là giới hạn thật. Một portfolio kỹ thuật chỉ có giá trị khi nói đúng điều mình đã và chưa làm được.

> Bước tiếp theo nếu có thời gian: thay worker bằng queue thật (SQS hoặc pg-boss), thêm batch replay, mã hoá secret at-rest, và đo tải 1.000 event/phút trên môi trường nhiều instance.

Tác giả HTK 0396241674 · 11/11/2026 · Dự án 07/10 trong portfolio full-stack.